MOCHA: Multi-Objective Co-Design using Hypernetwork Architectures

Anonymous Authors

***Abstract*— In this work, we present MOCHA, the first, to our knowledge, reinforcement learning based approach to computing a family of Pareto-optimal policies across the design space of a robot using a single network. Specifically, MOCHA leverages the hypernetwork architecture to learn a network that produces specialized network parameters optimized for a given objective and parameterized robot design; we term this a multi-objective design hypernetwork (MDH). We demonstrate the capabilities of MDHs to represent a complex family of design-dependent strategies on two distinct robot morphologies, each with six design dimensions and across 2-3 objectives. Moreover, we propose an approach for efficiently producing a *Design Pareto set* using evolutionary search of the learned policy network, generating the optimal design-policy combination for each objective prioritization. Lastly, we provide an efficient method for computing *generalist robot designs* which achieve the best cumulative performance across the entire set of objectives.**

I. Introduction

Robot co-design seeks to holistically consider the coupled effects of mechanical and control parameters on overall performance of robotic systems in an algorithmic fashion. By jointly optimizing the mechanics and control, co-design algorithms can achieve higher system performance than separately designing each. Traditional robot co-design treats the relationship between design and control parameters as a bi-level optimization problem, in which the outer loop selects a design and the inner loop selects a controller [1]–[4]. However, this architecture poses a challenge when using learning-based control policies, since each candidate design requires its own time-consuming training run.

One simple approach that has been demonstrated for reinforcement learning (RL) based co-design is to train a *universal policy* (UP) conditioned on a description of the robot design [5]–[7]. This flattens the bi-level optimization problem into a single loop that searches the design space for high-quality designs, guided by the policy rollouts or a universal value function. However, this results in policy behavior being tied to a specific pre-defined reward function. Once trained, the agent can only express behavior consistent with the reward function it was trained on, posing a challenge when the task prioritization is not exactly known beforehand.

In contrast, Multi-Objective RL (MORL) measures performance across multiple objectives with a vector-valued reward function. The goal of MORL is to learn a Pareto set of policies that optimally trade off across objectives. Explicitly, policies are only included in the Pareto set if no other policy can outperform it on an objective without sacrificing performance in another. By considering a Pareto set of policies instead of a single policy, one can explore the space of possible behaviors and train a single agent to complete multiple tasks without retraining. However, MORL has only been applied to robots with a fixed morphology and has not yet been extended to design-conditioned policies.

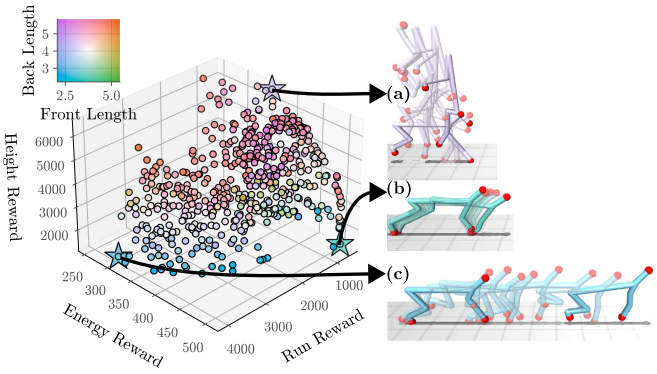


Fig. 1: By learning a control policy over multiple objectives and designs, our algorithm is capable of finding the optimal design for a given objective prioritization. For example, (a) is optimized for jumping, (b) is optimized for energy consumption, and (c) is optimized for running speed.

Our approach, titled MOCHA, combines the benefits of MORL and design-conditioned policies to train a *Multi-Objective Universal Policy* (MO-UP), whose value function and control policy optimally trade-off across multiple objectives. We argue that this combination unlocks the true potential of co-design by allowing a designer to explore the relationship between design and performance across competing objectives, and appropriately manage trade-offs.

We construct the MO-UP as a pair of value and policy hypernetworks, motivated by their success in expressing diverse families of policies in Multi-Objective Learning and Meta-Learning [8], [9]. Rather than representing the entire family of behaviors with a single fixed set of network parameters, a hypernetwork generates the parameters of a policy or value network conditioned on a context vector. This provides a parameter-efficient mechanism for representing a continuous family of specialized controllers across robot designs and objective trade-offs. We call our specific class of hypernetwork a *Multi-Objective Design Hypernetwork* (MDH). MOCHA also introduces an algorithm to search the input space of the MDH, producing a *Design Pareto frontier*, that is, a Pareto frontier of design-policy pairs, demonstrating its value in design space exploration.

The individual contributions of this work are as follows:

1) A demonstration of the hypernetwork architecture's capability to represent a family of Multi-Objective policies across the design space of a robot.
2) An algorithm that uses a hypernetwork as the Multi-

Objective Universal Policy to compute a Pareto set of design-policy pairs.
3) An algorithm for efficiently finding a *generalist* robot design, which achieves the best cumulative performance on all objectives.
4) An open-source, GPU-compatible implementation of the algorithms and benchmarks for Multi-Objective Co-design.

II. Related Work

Our work applies concepts from MORL and Multi-Domain RL to generate a policy that can be re-used for any design within a parameterized design space. This policy is embedded in a co-design algorithm to generate a Pareto set of designs. In this section, we will discuss existing methods to solve related problems.

A. Multi-Objective Reinforcement Learning

Unlike traditional RL, which maximizes a single scalar reward, MORL instead considers a vector-valued reward function consisting of a number of reward components. In most cases, the reward components cannot all be simultaneously maximized, as there are trade-offs between the objectives. Thus, MORL learns a Pareto set of policies, which optimally trade off across the objectives.

A number of algorithms exist to learn a continuous representation of the Pareto set. AMOR [10] and PD-MORL [11] both achieve this by learning a single policy (represented as a neural network) conditioned on the trade-off. However, both suffer from long training times, potentially due to the challenge of using a single neural network to represent both a policy distribution over the complex set of trade-offs and a robust control strategy.

Researchers have turned to hypernetworks, neural networks that output other networks conditioned on a context vector, as an alternative to represent the continuous Pareto set of policies. HYPER-MORL [12], PSL-MORL [13], and MORLAX [8] demonstrate that hypernetworks are capable of representing these policy families in a parameter-efficient manner. MORLAX connects hypernetwork-based MORL with GPU acceleration in JAX [14] to train Multi-Objective Policies using a Multi-Objective formulation of Proximal Policy Optimization (PPO). In this work, we build upon the MORLAX algorithm to train a multi-objective policy hypernetwork across the design space of a robot.

B. Multi-Domain Reinforcement Learning

The goal of Multi-Domain RL is to train a policy on a set of domains and achieve zero- or few-shot transfer to domains not seen in training. In the context of robot co-design, this enables a single reusable policy to be used to evaluate different robot designs.

Luo et al. and Feng et al. train controllers to generalize to a class of quadruped robots through randomization of the morphological parameters (mass and length) [5], [6]. This aligns with the broader paradigm of Hardware-Conditioned Policies (HCPs), where a single neural network takes explicit physical parameters (e.g., link masses, lengths) as inputs to adapt its control strategy across diverse robot configurations [15], [16]. In these settings, the robots are limited to a fixed kinematic structure, keeping the observation and action space constant across designs. To resole this, Bohlinger et al. leveraged an encoder-decoder architecture with a shared backbone to accommodate robots with differing observation and action spaces [7].

Hypernetworks have also demonstrated effectiveness in multi-domain generalization, particularly in the related problem setting of meta-learning. Rezaei-Shoshtari et al. used supervised learning to train a hypernetwork on a dataset of trajectories obtained by sampling pre-trained RL policies, enabling zero-shot transfer to unseen tasks and reward weights [17]. Additionally, Beck et al. trained a recurrent hypernetwork in an end-to-end fashion to represent a family of solutions to a Meta-RL problem [18]. The authors demonstrate that hypernetworks can prevent interference between distinct tasks while sharing parameters as necessary, resulting in diversity and parameter efficiency; properties which are leveraged in our work.

C. Co-Design

Jointly selecting mechanical and control parameters of a robot is a well-studied problem, beginning with bilevel optimization-based approaches [1], [19]. Recent work formulates the co-design problem directly as an RL problem. These include leveraging evolutionary algorithms [2], [20], or a co-trained designer policy [21], [22] to optimize morphology while using RL to train the control policy. Other methods utilize graph neural network-based heuristics [23], or an RL policy's value function [24], [25], as a proxy for design quality in a downstream design search algorithm. These approaches all use networks which take the design as input and are limited to single-objective learning tasks.

One of the few examples of multi-objective co-design is Multi-Objective Graph Heuristic search, which integrates multi-objective learning with co-design to analyze the Pareto set of robot designs [26]. This approach learns a heuristic function inspired by critic networks in RL to guide the design search, but expresses the control policy using model-predictive control. Similarly, Kim et al. proposed Multi-Objective Bilevel Bayesian Optimization (MO-BBO) to capture the Pareto set of robot morphologies and behaviors, though their approach relies on Bayesian optimization and has not been demonstrated for high-dimensional control parameterizations. In our work, we investigate hypernetworks as an alternative to design-conditioned policy networks in order to enable multi-objective robot co-design with RL-based control.

III. Method

A. Problem Setup

Consider a normalized design space $\mathcal{D} = [0, 1]^{n_d}$ representing the n_d-dimensional space of valid robot morphology parameters. We generalize the Multi-Objective Markov Decision Process (MOMDP) framework from MORL to be

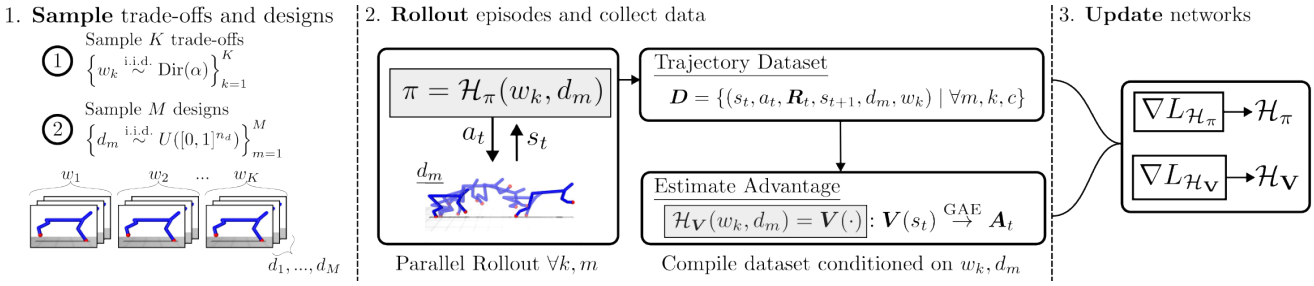


Fig. 2: The MO-UP Hypernetwork is trained by first sampling M designs and K trade-offs. The rewards obtained from rolling out these designs and trade-offs are used to update the value and policy networks using Multi-Objective PPO.

a design-aware MOMDP (DA-MOMDP) $(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{D}, \boldsymbol{R}, \gamma)$, where $\mathcal{S}$ is a set of states, $\mathcal{A}$ is a set of actions, $\mathcal{T}(s' \mid s, a, d)$ is the probability of transitioning from state $s \in \mathcal{S}$ to state $s' \in \mathcal{S}$ given action $a \in \mathcal{A}$ and design $d \in \mathcal{D}$, and $\gamma \in [0, 1)$ is a discount factor.

The vector-valued reward function $\boldsymbol{R} : S \times A \rightarrow \mathbb{R}^{n_r}$ is composed of n_r reward components, each describing a different objective. The trade-off vector w represents a relative prioritization of reward components and is drawn from Δ^{n_r-1}, the $(n_r - 1)$-dimensional simplex.

B. Hypernetwork Architecture

We represent the family of policies and value functions across the design and trade-off space using two hypernetworks: the Actor hypernetwork and Critic hypernetwork. Respectively,

$$\mathcal{H}_\pi : \Delta^{n_r-1} \times \mathcal{D} \rightarrow \Theta_\pi, \quad \mathcal{H}_V : \Delta^{n_r-1} \times \mathcal{D} \rightarrow \Theta_V, \quad (1)$$

which learn to predict the parameters of the policy and value neural networks with parameter spaces Θ_π and Θ_V, respectively. These functions are represented in the affine form

$$\mathcal{H}_{\pi,V}(w, d) = W_{\pi,V} f_{\pi,V}(w, d) + b_{\pi,V} \quad (2)$$

where $f_{(\cdot)}$ is a nonlinear function (represented as a neural network) termed the *task encoder* that maps a context vector comprised of w and d into a higher-dimensional task embedding. The task embedding is then transformed into the parameter space of the base networks through the matrix $W_{(\cdot)}$ and column vector $b_{(\cdot)}$. When successfully trained, this architecture learns a lower-dimensional manifold on which the set of relevant policies live, and indexes policies from this manifold using the corresponding context.

C. Multi-Objective Reinforcement Learning

The goal of MORL is to learn a policy that maximizes $w^T \boldsymbol{J}_\pi$, where

$$\boldsymbol{J}_\pi = \mathbb{E}_{s_0 \sim \mathcal{S}_0}[\boldsymbol{V}_\pi(s_0)] \in \mathbb{R}^{n_r} \quad (3)$$

$\boldsymbol{J}_\pi$ represents the expected return of policy π over initial state distribution $\mathcal{S}_0$, and $\boldsymbol{V}_\pi(s_0)$ is the vector-valued value function evaluated at the initial state $s_0 \in \mathcal{S}_0$, defined as

$$\boldsymbol{V}_\pi(s) = \mathbb{E}\Bigg[\sum_{t=0}^{\infty} \gamma^t \boldsymbol{R}(z_t, \pi(z_t)) \mid z_t \sim \mathcal{T}(z_{t-1}, \pi(z_{t-1}), d), z_0 = s\Bigg] \in \mathbb{R}^{n_r}. \quad (4)$$

The formulation of this learning problem mirrors that of single-objective RL, except that the reward and value functions are vector-valued. The policy is learned through a multi-objective extension of the Proximal Policy Optimization algorithm [27].

Since it may not be possible for a single policy to maximize all components of $\boldsymbol{J}_\pi$, the goal is to instead find a set of Pareto-optimal policies which optimally trade-off the objectives. Pareto optimality is determined by the notion of *Pareto dominance*: a policy π Pareto dominates a policy π' if $\boldsymbol{J}_\pi \geq \boldsymbol{J}_{\pi'}$ across every component, and $\boldsymbol{J}_\pi \neq \boldsymbol{J}_{\pi'}$ for at least one component. A policy is *Pareto optimal* if it is not dominated by any other policies. The set of all such optimal policies forms the *Pareto set*, and their corresponding rewards make up the *Pareto frontier*.

We measure the quality of a Pareto frontier by computing its *hypervolume* [28], defined as the volume enclosed by the Pareto frontier with respect to a given reference point (taken to be $\mathbf{0}$ in our implementation), and denote it as $\Gamma : \Pi \rightarrow \mathbb{R}$, where $\Pi \subset \Theta_\pi$ is a set of policies. Note that because we choose the reference point to be the origin, all rewards must be positive. We also measure the *spacing* of the Pareto frontier, which is the standard deviation of the distance between each point on the Pareto frontier and its nearest neighbor (in the reward space). A high quality Pareto frontier has high hypervolume and low spacing.

D. Multi-Objective Design-Conditioned Hypernetwork

We propose a Multi-Objective Design-Conditioned Hypernetwork (MDH) architecture to represent a MO-UP, trained through the following procedure, and illustrated in Figure 2. This approach builds upon MORL by training a policy to maximize $w^T \boldsymbol{R}$ over every design in $\mathcal{D}$.

1) Initialization: The algorithm is initialized with M, the number of sampled designs, K, the number of sampled trade-offs, and C, the number of trials for each trade-off-design pair. The MDH weight matrices $W_{(\cdot)}$ are initialized to all

zeros, and the weight vectors $b_{(\cdot)}$ are initialized randomly using Kaiming initialization [29]. The weights of the task encoders are also initialized with Kaiming initialization. Empirically, we find that this initialization helps to promote diversity in the hypernetwork policies, corroborating [9].

2) Sampling and Rollout: Each training epoch begins with drawing a set of K trade-offs $\{w_k\}_{k=1}^{K}$. The first $K - n_r$ are sampled from a Dirichlet distribution with concentration parameters $\alpha_i = 1, i = 1, ..., n_r$, and the remaining are taken to be the corners of Δ^{n_r-1}, (e.g. for $n_r = 3$, $[1,0,0], [0,1,0], [0,0,1]$). This guarantees that the extreme tradeoffs are explored. Then, M designs $\{d_m\}_{m=1}^{M}$ are sampled uniformly from $\mathcal{D}$ using a Sobol sequence [30]. This grid of design-trade-off combinations is used to instantiate $M \cdot K \cdot C$ parallel environments.

Each environment is simulated according to its dynamics (parameterized by d_m) and the policy $\pi_{w,d} = \mathcal{H}_\pi(w_k, d_m)$. These rollouts are collected into a dataset $\boldsymbol{D}$ of design-conditioned state-action transitions, with each entry of the dataset defined as

$$\boldsymbol{D}^{(m,k,c)} = \left(s_t^{(m,k,c)}, a_t^{(m,k,c)}, \boldsymbol{R}_t^{(m,k,c)}, s_{t+1}^{(m,k,c)}, w_k, d_m\right), \tag{5}$$

and m, k, and c, denoting the design, trade-off, and trial indices, respectively.

3) Gradient Update: The collected dataset $\boldsymbol{D}$ is used to update the MDH weights. First, we estimate a vector-valued advantage for each state-action trajectory $\boldsymbol{A}_t \in \mathbb{R}^{n_r}$ by applying Generalized Advantage Estimation independently to each objective [31]. Traditionally, these advantages are shuffled, grouped into batches, and normalized by the batch mean and standard deviation before being used to compute the policy gradient.

We observe that naively normalizing the advantages without regard for the design results in mode collapse, where the policy learns to take the same or similar actions irrespective of the design. To avoid this, we group the set of advantages according to their associated design, and normalize each advantage relative to only the others in its group. The group-wise normalized advantages are used to update the hypernetworks by applying stochastic gradient descent to minimize the following losses (dataset indices (m, k, c) dropped for readability):

$$L_{\mathcal{H}_\pi} = -\mathbb{E}_{(s_t,a_t,w,d)\sim\boldsymbol{D}}\left[r_t(\theta)\, w \cdot \boldsymbol{A}_t\right], \tag{6}$$

$$L_{\mathcal{H}_V} = \mathbb{E}_{(s_t,w,d)\sim\boldsymbol{D}}\left[\left\|\boldsymbol{V}_\phi(s_t) - \hat{\boldsymbol{V}}_t\right\|_2^2\right] \tag{7}$$

$$\text{where } r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta,\text{old}}(a_t \mid s_t)}, \tag{8}$$

$$\theta = \mathcal{H}_\pi(w, d),\ \phi = \mathcal{H}_V(w, d). \tag{9}$$

Here, $r_t(\theta)$ denotes the probability ratio of a particular action taken by the policy with parameters θ obtained from the hypernetwork with the associated context (w, d), $\boldsymbol{V}_\phi$ denotes the estimate produced by the value network with parameters ϕ also obtained from the hypernetwork, and $\hat{\boldsymbol{V}}$ denotes the value computed from the rollouts in the dataset. In practice, the clipped surrogate PPO loss is used to stabilize training, as in standard PPO implementations [32].

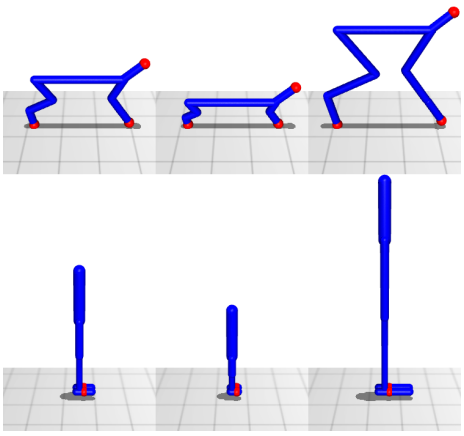


Fig. 3: Some examples of procedurally generated Cheetah and Walker robots.

E. Computing the Design Pareto Frontier

We wish to compute the *Design-Pareto Set*, $\mathcal{P}$, which associates each trade-off $w^\star \in \Delta^{n_r-1}$ with the design $d^\star$ and conditioning trade-off $\tilde{w}$ whose induced policy performs best under $w^\star$:

$$\mathcal{P} = \Big\{(w^\star,\, \tilde{w},\, d^\star) \;\Big|\; w^\star \in \Delta^{n_r-1} \text{ and } (\tilde{w}, d^\star) = \arg\max_{w,d} (w^\star)^\top R\big(\mathcal{H}(w,d)\big)\Big\}. \tag{10}$$

Here $R : \theta_\pi \to \mathbb{R}^{n_r}$ denotes the vector of expected cumulative rewards obtained by policy π. We deliberately search over the conditioning weight $\tilde{w}$ independently of the evaluation weight $w^\star$. For a given $w^\star$, the policy $\mathcal{H}(\tilde{w}, d)$ obtained from some $\tilde{w} \neq w^\star$ may attain a higher scalarized return $(w^\star)^\top R$ than the policy produced by $w^\star$, $\mathcal{H}(w^\star, d)$. A hypernetwork trained to optimality under the MO-PPO objective would theoretically satisfy $\tilde{w} = w^\star$, since conditioning on $w^\star$ would, by definition, yield the policy maximizing $(w^\star)^\top R$. In practice this holds approximately, and the conditioning $\tilde{w}$ that produces the best performing policy is often nearly identical to $w^\star$.

While $\mathcal{P}$ is by construction a continuous set over $w^\star$, analytically computing it is infeasible. Prior approaches approximate $\mathcal{P}$ via random sampling [8], [10], however this quickly becomes computationally infeasible in moderate to high dimensional design and tradeoff spaces. Instead, we employ evolutionary search to find a dense collection of 3-tuples $(w^\star, \tilde{w}, d^\star)$ that approximate $\mathcal{P}$.

Specifically, we utilize Non-dominated Sorting Genetic Algorithm (NSGA) III [33]. NSGA-III is a computationally-efficient, evolutionary multi-objective optimization program that finds high-quality Pareto fronts by optimizing for both hypervolume and coverage of the objective space. During optimization, NSGA-III samples points in batches, making

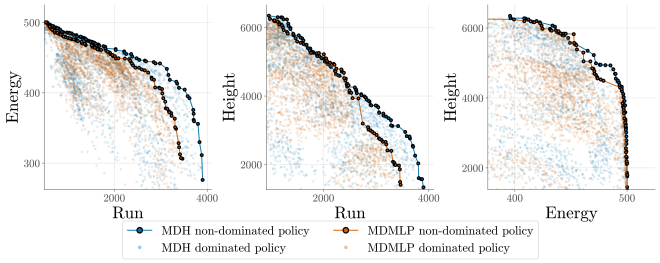


Fig. 4: The Design Pareto frontier of the Cheetah sliced along the Run-Energy, Run-Height, and Energy-Height axes.

Network	Env.	Hypervolume	Spacing	Train Time
MDH	Cheetah	**6.838e9**	**5.195e-2**	**2h 6m**
MDMLP	Cheetah	5.353e9	0.1224	6h 20m
MDH	Walker	**2.715e6**	**5.656e-2**	**1h 48m**
MDMLP	Walker	2.576e6	9.566e-2	5h 55m

TABLE I: Performance of the hypernetwork algorithm on Walker and Cheetah Environments compared to the MLP baseline

it an ideal candidate algorithm for our approach, which already leverages massively parallelized simulation. Unlike classic multi-objective optimization where the sample space is restricted to a hyperrectangle, our algorithm searches over the design and trade-off space, where the trade-off is restricted to the simplex. Thus, we implement a simplex repair method, which projects sampled points back onto the simplex after genetic mixing. When combined with parallelized simulation, we achieve denser, higher quality Design Pareto sets than by random sampling.

In addition to using the Design Pareto front to identify the best design $d^\star$ and control strategy $\mathcal{H}(\tilde{w}, d^\star)$ for a particular trade off $w^\star$, we also utilize its hypervolume as an indicator of algorithm performance and confirmation of convergence (i.e. when $\Gamma(\mathcal{H}_\pi)$ plateaus).

F. Finding a Generalist Design

After applying the method in Sec. III-E, points on the Design Pareto frontier correspond to design-policy combinations that maximize a particular objective prioritization. However, there are situations where a generalist robot design, capable of performing well on all objectives, may be preferred. For example, if only one robot can be fabricated before being sent on an unknown mission, that design should be capable of handling a large volume of potential missions.

Thus, we define a generalist design as one whose associated multi-objective policy has a high hypervolume. In other words, it achieves high reward for a large number of possible trade-offs. We then formulate an optimization problem to find the single design d^g whose associated multi-objective policy has the highest hypervolume. Formally,

$$d^g = \underset{d \in \mathcal{D}}{\text{argmax}}\ \Gamma(\mathcal{H}_\pi(w, d)). \tag{11}$$

Computing the Design Pareto set's hypervolume is computationally expensive and non-differentiable due to the need to forward-simulate the policy across many sampled

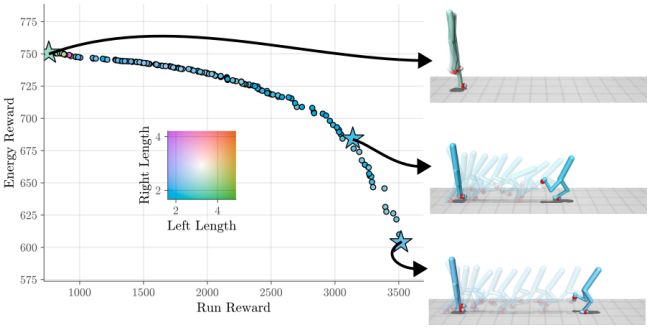


Fig. 5: Design Pareto frontier for the 2-objective Walker, and some rollouts of Pareto-optimal designs.

tradeoffs and designs. Therefore, we solve the above optimization problem using Trust Region Bayesian Optimization (TuRBO), a sample-efficient, zeroth-order optimizer [34].

IV. Results

To analyze the performance of MOCHA, we ask three guiding questions. First, how good is the MDH at representing optimal policies for any given design-trade-off pair? Second, does the hypernetwork architecture offer improvements over a multi-objective, design-conditioned multi-layer perceptron (MLP) policy? Third, how can a practitioner interpret the resulting Pareto set to understand the design space?

A. Implementation Details

We evaluate our approach on a class of robots based on environments from the well-known `dm_control` suite [35]. In our open-source codebase[1], we implement procedurally generated versions of the Cheetah and Walker environments, some examples of which are shown in Figure 3.

For our Cheetah environment, we define a 6-dimensional design space corresponding to each of the segments making up the front and back legs. The 6D normalized design space is affinely mapped onto the range $[0.5, 2]^6$ corresponding to a scaling of of each link relative to the link length of the original robot. In this environment, the reward consists of three components: (1) *Run*, a reward on the velocity in the +x direction, (2) *Height*, a reward on the height of the robot's hip, and (3) *Energy*, a reward for minimizing the total actuator force. Similarly, the Walker environment is comprised of a 6D design space corresponding to the lengths of the left and right leg's thigh, shin, and foot segments, formulated using the same scaling procedure as the Cheetah. The Walker receives Run and Energy rewards, formulated similarly to the Cheetah's.

We implement these environments using the MO-Playground library [8], which leverages MuJoCo JAX [36] for massive parallel computation. The Pareto frontier metrics (spacing and hypervolume) and NSGA-III algorithm are computed using the `pymoo` library [37], and TuRBO is implemented using the `botorch` library [38].

[1] https://anonymous.4open.science/r/codesign-E7CD/

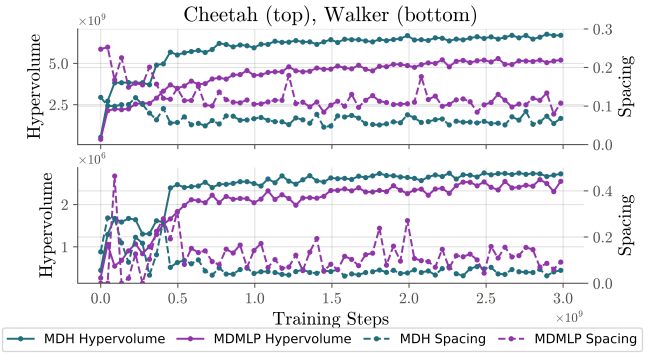


Fig. 6: Comparison of the Design Pareto Frontier metrics attained by a Hypernetwork and MLP policy for the Cheetah (top) and Walker (bottom) environments. The Hypernetwork achieves a higher hypervolume and lower spacing than the MLP in both environments.

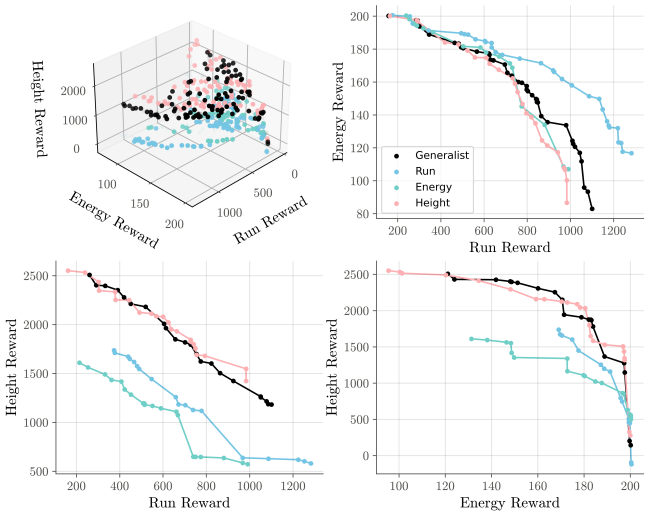


Fig. 7: Fixed-Design Pareto frontiers for the generalist Cheetah, and a few selected specialists. Note that the run-optimized design far outperforms the others at running and energy savings, and the generalist design performs approximately similar to the height-optimized design.

We train an MDH on each of these environments. The task encoder is comprised of 2 hidden layers of size $[1024, 1024]$ and maps the context (design and tradeoff) to a 128-dimensional task embedding. The policy and value hypernetworks each have 3 hidden layers of size $[256, 256, 128]$. Training was conducted on an NVIDIA RTX Pro 6000 Blackwell GPU. Further details can be found in the linked code repository.

B. Analysis of the Design Pareto Frontier

The 3D Design Pareto frontier for the Cheetah environment is shown in Figure 1. Sampling and simulating the design-policy combinations in the Pareto set show that the hypernetwork is capable of learning a diverse set of skills, such as jumping to maximize height, sitting to minimize energy, and running forward to maximize speed. As expected, designs with long legs achieve maximal height reward. Short-legged designs run fast since lower inertia allows them to oscillate back and forth faster. It is trivially easy for a policy to maximize the energy reward by doing nothing, so almost every design does well on this reward.

Similarly, we compute the 2-dimensional Design Pareto frontier for the Walker environment, and show it in Figure 5. Here, symmetric designs end up dominating on almost all reward combinations, except for a purely energy-maximizing strategy where one leg is much shorter than the other. This demonstrates the capability of our algorithm to discover the relevant subspaces of an otherwise intractable design space.

C. Comparison to Multi-Objective Design-Conditioned MLP Policy

In order to demonstrate the effectiveness of the hypernetwork architecture, we also train an MLP-based multi-objective universal policy (referred to as MDMLP) on the Cheetah and Walker environments. The MDMLP policy takes in the observation and context vectors, and outputs an action. It is comprised of 3 hidden layers, selected such that the total number of parameters is equal to that of the MDH. The MDMLP is trained with the same multi-objective PPO loss functions as the hypernetwork. A summary of the results of the comparison is provided in Table I, with a visualization of the obtained hypervolumes against the number of environment steps provided in Figure 6.

We train all networks for 3 billion timesteps. For Cheetah, the MDH achieved a 28% greater hypervolume than the MDMLP policy, while for the walker it achieved a 6% greater hypervolume. Slices of the Cheetah's Design Pareto frontiers obtained by the MDH and MDMLP are shown in figure 4. The MDMLP performs similarly to the MDH for the Height and Energy objectives but fails at Run, demonstrating the hypernetwork's superior ability to represent a diverse set of behaviors. Additionally, for both environments, the training time for the MDH was a third of that of the MDMLP. This difference arises because forward inference of the hypernetwork only occurs once per rollout, and the much smaller policy network is used every timestep. In contrast, the large MDMLP must be used for each inference at every timestep, requiring more computation and slowing down overall throughput.

D. Comparison of Generalist and Specialists

To demonstrate how a practitioner can analyze the Design Pareto frontier, we extract and test four morphologies of the Cheetah environment, with each fixed-design Pareto frontier shown in Figure 7. Specifically, we select three extreme "specialist" designs, each maximizing one specific objective, and one "generalist" design chosen as the solution to the optimization problem of Section III-F. These morphologies are illustrated in Figure 8. By evaluating all four morphologies across a selection of objectives, one can observe the relationship between a robot's specific physical traits,

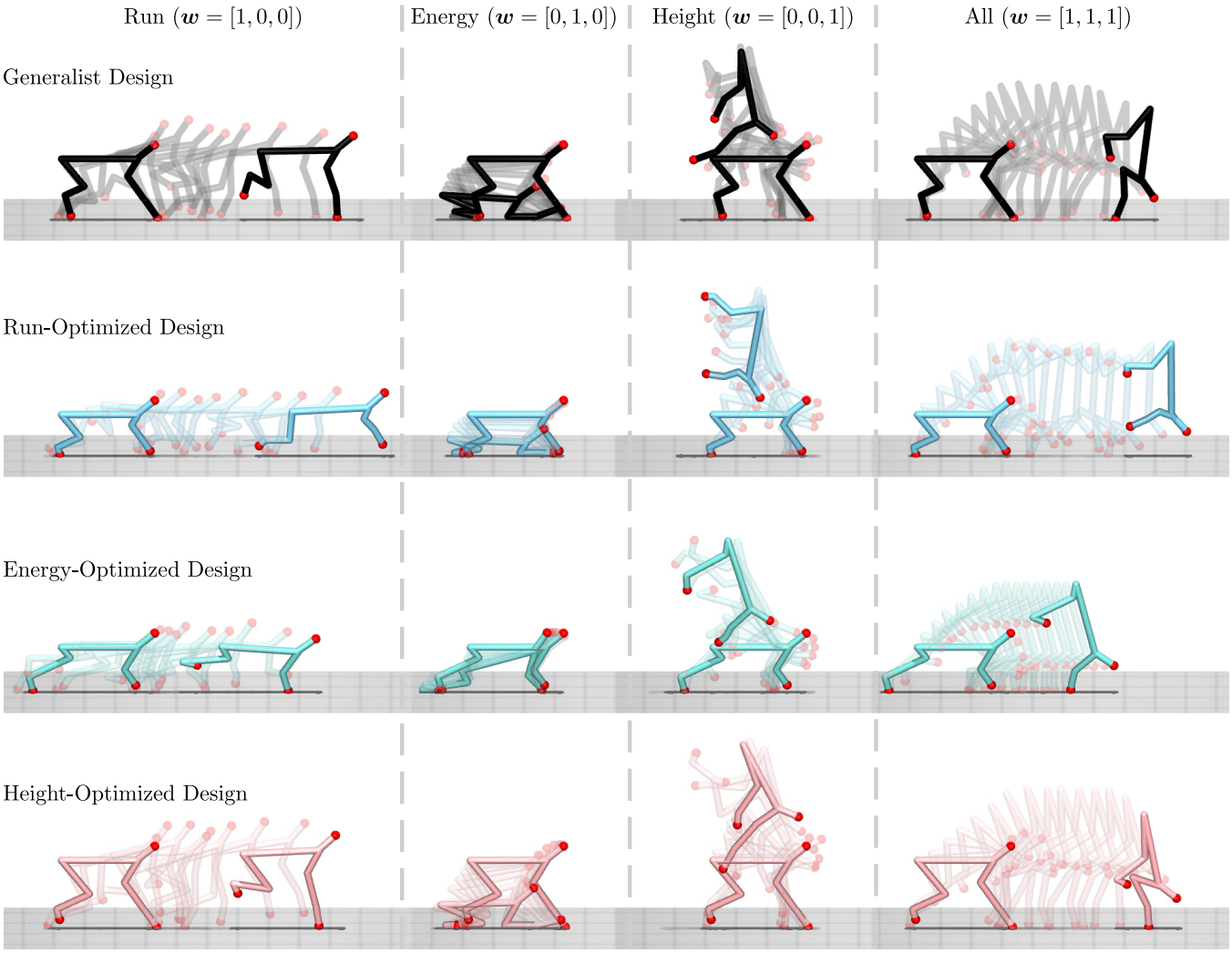


Fig. 8: Procedurally-generated generalist and specialist Cheetah robots obtained using MOCHA. The generalist robot maximizes the hypervolume of its associated Pareto frontier, while each task-optimized design maximizes the scalar reward according to its objective prioritization.

strategies taken by its control policy, and its performance on the task set. For example, we observe that short legs help the run-optimized design not only run faster, but save energy while doing so. Additionally, we see that the generalist design is morphologically similar and performs similarly to the height-optimized design. The discovery of this non-obvious generalist robot is made possible through the multi-objective problem formulation of MOCHA.

V. Limitations

There are a few weaknesses of our approach that can be addressed in future work. First, the universal policy may not be the most accurate way to score a design. If the universal policy does not discover optimal strategies, a suboptimal design may be over-rated. While this may be resolved by training a larger hypernetwork, more studies are required to better understand the scaling capability of hypernetworks.

Second, the design space representation must be isomorphic to an n_d-dimensional hypercube. This means our algorithm cannot explicitly represent complex design constraints (e.g. self-collision avoidance). One solution to address this would be to add a cost for design constraint violations to prevent these designs from being assigned to the Pareto set.

Lastly, our work does not consider sim-to-real transfer of the learned policies. It may be possible that some parts of the design space are more likely to exhibit good sim-to-real transfer. Including domain randomization and a "trust region" in the design space are exciting directions of future research.

VI. Conclusion and Future Work

In this paper, we introduce MOCHA, a framework for training and evaluating a Multi-Objective Design Hypernetwork, which is used in combination with multi-objective optimization as a foundation for multi-objective co-design. This architectural choice is motivated by the recent success of hypernetworks in Multi-Objective RL and Meta RL. Namely, we leverage the ability for hypernetworks to learn a diverse set of strategies while sharing information through a single policy representation, resulting in improved sample-efficiency. MOCHA efficiently learns a Pareto set of policy

and value network parameters for any design in a continuous design space, which provides a low-dimensional space to search when computing the optimal specialized design and policy for a given task. Combining these specialized Pareto sets results in a Design Pareto frontier, allowing a designer to analyze how each design parameter affects task performance across the set of objectives. Our framework also enables algorithmic computation of a generalist design, which achieves maximal performance across all objectives. We demonstrate MOCHA across two environments, both comprised of a 6-dimensional design space and multiple objectives. In both, the hypernetwork-based policy achieves a higher hypervolume in fewer training iterations and less wall-clock time. Overall, this work provides a promising first step at leveraging Multi-Objective Reinforcement Learning to co-design optimal robot designs and control policies.

VII. Acknowledgements

The authors used models from Anthropic and OpenAI to assist in the code implementation of MOCHA.

References

[1] S. Ha, S. Coros, A. Alspach, J. Kim, and K. Yamane, "Computational co-optimization of design parameters and motion trajectories for robotic systems," *The International Journal of Robotics Research*, vol. 37, no. 13-14, pp. 1521–1536, 2018.

[2] J. Bhatia, H. Jackson, Y. Tian, J. Xu, and W. Matusik, "Evolution gym: A large-scale benchmark for evolving soft robots," *Advances in Neural Information Processing Systems*, vol. 34, pp. 2201–2214, 2021.

[3] Y. Kim, Z. Pan, and K. Hauser, "MO-BBO: Multi-objective bilevel bayesian optimization for robot and behavior co-design," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9877–9883, IEEE, 2021.

[4] B. Yue, S. Xu, K. Jia, and G. Liu, "Toward humanoid brain-body co-design: Joint optimization of control and morphology for fall recovery," *arXiv preprint arXiv:2510.22336*, 2025.

[5] Z. Luo, Y. Dong, X. Li, R. Huang, Z. Shu, E. Xiao, and P. Lu, "Moral: Learning morphologically adaptive locomotion controller for quadrupedal robots on challenging terrains," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, pp. 4019–4026, 2024.

[6] G. Feng, H. Zhang, Z. Li, X. B. Peng, B. Basireddy, L. Yue, Z. Song, L. Yang, Y. Liu, K. Sreenath, and S. Levine, "Genloco: Generalized locomotion controllers for quadrupedal robots," 2022.

[7] N. Bohlinger, G. Czechmanowski, M. Krupka, P. Kicki, K. Walas, J. Peters, and D. Tateo, "One policy to run them all: an end-to-end learning approach to multi-embodiment locomotion," 2025.

[8] N. Janwani, E. Novoseller, V. J. Lawhern, and M. Tucker, "Mo-playground: Massively parallelized multi-objective reinforcement learning for robotics," 2026.

[9] J. Beck, M. T. Jackson, R. Vuorio, and S. Whiteson, "Hypernetworks in meta-reinforcement learning," 2022.

[10] L. N. Alegre, A. Serifi, R. Grandia, D. Müller, E. Knoop, and M. Bächer, "Amor: Adaptive character control through multi-objective reinforcement learning," in *Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers*, SIGGRAPH Conference Papers '25, p. 1–11, ACM, 2025.

[11] T. Basaklar, S. Gumussoy, and U. Y. Ogras, "Pd-morl: Preference-driven multi-objective reinforcement learning algorithm," 2023.

[12] T. Shu, K. Shang, C. Gong, Y. Nan, and H. Ishibuchi, "Learning pareto set for multi-objective continuous robot control," 2024.

[13] E. Liu, Y.-C. Wu, X. Huang, C. Gao, R.-J. Wang, K. Xue, and C. Qian, "Pareto set learning for multi-objective reinforcement learning," 2025.

[14] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018.

[15] W. Yu, J. Tan, C. K. Liu, and G. Turk, "Preparing for the unknown: Learning a universal policy with online system identification," *arXiv preprint arXiv:1702.02453*, 2017.

[16] T. Chen, A. Murali, and A. Gupta, "Hardware conditioned policies for multi-robot transfer learning," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[17] S. Rezaei-Shoshtari, C. Morissette, F. R. Hogan, G. Dudek, and D. Meger, "Hypernetworks for zero-shot transfer in reinforcement learning," 2023.

[18] J. Beck, R. Vuorio, Z. Xiong, and S. Whiteson, "Recurrent hypernetworks are surprisingly strong in meta-rl," 2023.

[19] A. B. Ghansah, J. Kim, M. Tucker, and A. D. Ames, "Humanoid robot co-design: Coupling hardware design with gait generation via hybrid zero dynamics," 2023.

[20] A. Gupta, S. Savarese, S. Ganguli, and L. Fei-Fei, "Embodied intelligence via learning and evolution," *Nature communications*, vol. 12, no. 1, p. 5721, 2021.

[21] Y. Yuan, Y. Song, Z. Luo, W. Sun, and K. Kitani, "Transform2act: Learning a transform-and-control policy for efficient agent design," 2022.

[22] C. Schaff and M. R. Walter, "N-limb: Neural limb optimization for efficient morphological design," 2022.

[23] A. Zhao, J. Xu, M. Konaković-Luković, J. Hughes, A. Spielberg, D. Rus, and W. Matusik, "Robogrammar: graph grammar for terrain-optimized robot design," *ACM Transactions on Graphics (TOG)*, vol. 39, no. 6, pp. 1–16, 2020.

[24] N. Bohlinger and J. Peters, "Shape your body: Value gradients for multi-embodiment robot design," 2026.

[25] K. S. Luck, H. B. Amor, and R. Calandra, "Data-efficient co-adaptation of morphology and behaviour with deep reinforcement learning," 2019.

[26] J. Xu, A. Spielberg, A. Zhao, D. Rus, and W. Matusik, "Multi-objective graph heuristic search for terrestrial robot design," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9863–9869, 2021.

[27] J. Xu, Y. Tian, P. Ma, D. Rus, S. Sueda, and W. Matusik, "Prediction-guided multi-objective reinforcement learning for continuous robot control," in *Proceedings of the 37th International Conference on Machine Learning* (H. D. III and A. Singh, eds.), vol. 119 of *Proceedings of Machine Learning Research*, pp. 10607–10616, PMLR, 13–18 Jul 2020.

[28] E. Zitzler and L. Thiele, "Multiobjective optimization using evolutionary algorithms—a comparative case study," in *International conference on parallel problem solving from nature*, pp. 292–301, Springer, 1998.

[29] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," 2015.

[30] I. Sobol', "On the distribution of points in a cube and the approximate evaluation of integrals," *USSR Computational Mathematics and Mathematical Physics*, vol. 7, no. 4, pp. 86–112, 1967.

[31] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," 2018.

[32] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, "Brax – a differentiable physics engine for large scale rigid body simulation," 2021.

[33] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part i: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[34] D. Eriksson, M. Pearce, J. R. Gardner, R. Turner, and M. Poloczek, "Scalable global optimization via local bayesian optimization," 2020.

[35] S. Tunyasuvunakool, A. Muldal, Y. Doron, S. Liu, S. Bohez, J. Merel, T. Erez, T. Lillicrap, N. Heess, and Y. Tassa, "dm_control: Software and tasks for continuous control," *Software Impacts*, vol. 6, p. 100022, 2020.

[36] K. Zakka, B. Tabanpour, Q. Liao, M. Haiderbhai, S. Holt, J. Y. Luo, A. Allshire, E. Frey, K. Sreenath, L. A. Kahrs, C. Sferrazza, Y. Tassa, and P. Abbeel, "Mujoco playground: An open-source framework for gpu-accelerated robot learning and sim-to-real transfer.," 2025.

[37] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89497–89509, 2020.

[38] M. Balandat, B. Karrer, D. R. Jiang, S. Daulton, B. Letham, A. G. Wilson, and E. Bakshy, "BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization," in *Advances in Neural Information Processing Systems 33*, 2020.